A Cell-to-Symbol Tally Approach for Generalised Sudoku

Shiv Bali

shiv.somappa@gmail.com

Preprint v01 — April 2026. Comments welcome.

Version 2 of this preprint will be released with fully independent solver data, including runtime comparisons against DLX and SAT baselines across ranks 3–20.

Abstract

This paper presents the Cell-to-Symbol Tally (CTST) method, a lightweight constraint-propagation framework for generalised Sudoku that operates directly on candidate structures. CTST combines eight deterministic filtering rules, a prediction phase, inconsistency-based root-cause detection, and a hint-preserving rollback mechanism. Prediction is not restricted to cells with small candidate sets: at each deterministic fixed point the solver selects the cell with the fewest remaining candidates, whatever that size may be. Rules 5–6 provide full tight-subset detection across all sizes. Rule 8 introduces a near-tight filtering procedure generalised to any surplus degree n ($s - c = n$): when exactly n cells in G each carry exactly two singleton-occurrence symbols, shared symbols are removed from those n cells and from all right-side cells. The general theorem holds for all n; the rule is implemented for $n = 1$ and $n = 2$. Rule 7 provides complementary symbol-first filtering; its structural contribution, including downstream cascade effects from the first-trigger pipeline, is measured through the ablation study in §8.3. Structural rules (Rules 1,5,6,8) perform a dual role, combining candidate elimination with immediate inconsistency detection, while other rules act primarily as propagators.

The contribution lies in explicit, low-cost operationalisation of structural regimes—particularly the near-tight case—rather than increased deductive power beyond generalised arc consistency. Across experimental data with K ranging 1–14, an empirical runtime scaling of $t \sim K^{3.17}$ is observed ($R^2 = 0.97$).

Keywords: Sudoku, constraint propagation, NP-complete, near-tight subsets, s−c=n analysis, bound-sets filter, ranks 3–20, predict-and-rectify solver, first-trigger priority pipeline.

Note (v01): *This is version 1 of this preprint. Version 2 will be released with fully independent solver data, including runtime comparisons against DLX and SAT baselines across ranks 3–20.*

---

1. Introduction

Sudoku and its generalisations are among the most widely studied combinatorial puzzles in computer science. Beyond recreational interest, they serve as natural benchmarks for constraint-satisfaction solvers: the 9×9 variant is accessible enough to reason about by hand, yet the generalised N×N decision problem is NP-complete [1].

1.1 Scope

CTST is designed as a general-purpose solver for N×N Sudoku grids ($N = n^2$, $n \geq 3$). The deterministic phase (Rules 1–8) exhausts all deduction available through constraint propagation before prediction is invoked. At each deterministic fixed point the solver selects the unresolved cell with the smallest candidate set—regardless of how many candidates that cell holds—and guesses one value, then propagates. This unrestricted prediction policy ensures the solver never halts due to a candidate-count threshold: it continues until the puzzle is solved or all branches are exhausted.

The goal is a practical, lightweight solver that directly exploits Sudoku-specific structure rather than globally complex mechanisms. CTST isolates specific structural regimes—particularly the near-tight case ($s - c = n$)—and provides direct procedures for each. A runtime comparison against independent solver baselines (DLX, SAT) is deferred to a subsequent version pending completion of that evaluation.

1.2 Motivation for Dedicated Lightweight Algorithms

Existing alldifferent filtering algorithms—Régin [6], Leconte [9], Puget [11], Lopez-Ortiz et al. [12]—provide strong guarantees but work through globally complex mechanisms. Two regimes appear underexplored in terms of dedicated lightweight treatment.

The first is the near-tight regime ($s - c = n$ for positive integer n). The second is symbol-first tight detection: no lightweight accumulation algorithm proceeding directly from symbol occurrence counts was identified in the surveyed literature.

1.3 Contributions

- **Two-pass tally (Rules 5–6):** full tight-subset detection for all sizes, unifying naked and hidden subset detection. Rule 3 retained as fast-path optimisation.
- **Rule 8 — generalised near-tight analysis ($s - c = n$):** when exactly n cells in G each carry exactly two singleton-occurrence symbols, shared symbols are eliminated from those n cells and all right-side cells. This constitutes a general theorem valid for all n, with implementation covering $n = 1$ and $n = 2$.
- **Rule 7 — Bound-Sets Filter:** symbol-first accumulation; removes candidates from inside bound cells; complementary to Rules 5–6 (cell-first, outside elimination). Even a single elimination by Rule 7 restarts the full pipeline, producing downstream cascade effects in highly constrained candidate structures.
- **Unrestricted prediction:** the solver selects the cell with the fewest candidates at each fixed point, with no upper bound on candidate set size, ensuring completeness for arbitrary generalised Sudoku instances within the available branching budget.
- **Hint-preserving rollback:** predicted symbols retained as hints, reducing empirical branching (see §7.5 and §8).
- **Inconsistency-based root-cause error cell detection:** solver traces triggerLv records and backjumps directly to the originating prediction.
- **Rules 5–6 Left-Side Singleton Collision Error:** logical impossibility signalled immediately when a tight group contains a cell with two or more singleton-occurrence symbols.

The contribution is operationalisation: direct lightweight procedures for structural regimes previously handled only implicitly.

2. Related Work

2.1 Complexity

Yato and Seta [1] proved that the generalised Sudoku decision problem is NP-complete. The CTST design responds to this by maximising the deterministic phase (Rules 1–8) to reduce prediction frequency and depth. The deterministic phase has an $O(N^5)$ per-pass cost (§7.4); the prediction phase remains exponential in the theoretical worst case.

2.2 Subset Propagation and the Alldifferent Constraint

Régin [6] proved that alldiff(x_1, …, x_n) is generalised arc-consistent (GAC) if and only if every edge in its variable-value bipartite graph belongs to some maximum matching. The SCC partition implicitly identifies all tight subsets of arbitrary symbol sets; Gent et al. [15] provide a detailed treatment of GAC for alldifferent via SCC decomposition. Van Hoeve's survey [10] provides a comprehensive treatment.

Leconte [9] introduced range consistency using Hall intervals in $O(N^2)$. Puget [11] improved this to $O(N \log N)$ bounds consistency. Lopez-Ortiz et al. [12] gave a practical improvement. None detect tight subsets of arbitrary, non-contiguous symbol sets.

Rule 7 in context. Régin's SCC decomposition subsumes Rule 7's detection power. However, Rules 5–6 eliminate from outside tight groups (cell-first) while Rule 7 eliminates from inside bound cells (symbol-first); the two are complementary in the current implementation. Rule 7 fires only at the residual fixed point of all preceding rules; its contribution is measured through the combined ablation study in §8.3.

2.3 The Near-Tight Regime (s − c = n)

All alldifferent filtering algorithms identified in the surveyed literature primarily target tight subsets (s − c = 0) via Hall's condition. Near-tight configurations (s − c = n for positive integer n) are typically handled implicitly within full generalised arc-consistency (GAC) propagation rather than through a dedicated procedure.

To our knowledge, existing literature does not isolate the near-tight regime (s − c = n) as an explicit operational case in this form. We do not claim new deductive power beyond GAC; the contribution lies in providing a lightweight, direct operationalisation of this regime.

Rule 8 addresses the near-tight case through two phases. In the Resolve phase, shared symbols are removed from right-side cells once the singleton-occurrence pattern is established. In the Error phase, an impossible configuration triggers immediate rollback. Rule 8 is implemented for s − c = 1 (one surplus symbol, one key cell with two singletons) and s − c = 2 (two surplus symbols, two key cells each with two singletons). Extensions to s − c ≥ 3 follow the same pattern but are rare in practice.

The deductions are subsumed by GAC, but isolating this regime enables early error detection and lightweight filtering without requiring global matching machinery.

2.4 Pointing Pairs and Box-Line Reduction

Pointing pairs and box-line reduction (Rule 4) appear in Simonis [8] and extensively in puzzle-solving literature. The CTST method generalises this to any N×N grid.

2.5 Algorithm X, CDCL, and SAT Encodings

CDCL [4] informs the hint rollback design. Dechter [13] establishes the conceptual framework of conflict-directed backjumping and no-good recording, which is the ancestor of the triggerLv-based root-cause detection in §6.3. Lynce and Ouaknine [14] demonstrated that SAT encodings of Sudoku with extended clauses allow unit propagation alone to solve many puzzles. Norvig [2]

provides a practical constraint-propagation approach to 9×9 Sudoku that motivates the lightweight propagation design of CTST.

3. Definitions

All definitions apply to an N×N grid with $N = n^2$ for integer $n \geq 3$.

Term	Definition
Definition 1 (Grid)	Block size $n \geq 3$; grid size $N = n^2$. Cells (i,j), $1 \leq i,j \leq N$. A unit is any row, column, or block; there are 3N units in total.
Definition 2 (Candidate set)	$C(i,j) \subseteq \{1,\ldots,N\}$, the set of remaining candidates for cell (i,j). When $\lvert C(i,j)\rvert = 1$, the cell is resolved (naked single).
Definition 3 (Tight subset)	A set S of unresolved cells in unit U is tight if $\lvert \cup\{ C(i,j) : (i,j) \in S \}\rvert = \lvert S\rvert$ — the union of their candidate sets is exactly as large as the number of cells (Hall equality).
Definition 3a (s−c surplus metric)	For a group G of c cells with $s = \lvert \cup\{ C(i,j) : (i,j) \in G \}\rvert$ distinct symbols, the surplus metric is $s - c$. A tight subset has $s - c = 0$; a near-tight group has $s - c = n$ for positive integer n.
Definition 4 (Near-tight group)	A group G is near-tight if $s - c = n$ for some positive integer n. Rule 8 is implemented for n = 1 and n = 2; the general theorem holds for all n (§4.4.8).
Definition 5 (Solo(c))	For a near-tight group G and cell c, solo(c) counts the number of singleton-occurrence symbols in c (symbols appearing in exactly one group cell, namely c).
Definition 6 (Key cell and K)	A key cell is an unresolved cell selected for prediction at a deterministic fixed point on the basis of having the smallest candidate set. K denotes the total number of prediction steps.
Definition 6a (p_ET)	A 3D candidate table of size $N^2 \times (N+4)$ on which all deterministic filtering Rules 1–8 are applied. p_ET tracks the complete filtered candidate state across the full solve.
Definition 6b (p_PS)	A 3D candidate table of size $N^2 \times (N+4)$ on which only prediction updates are applied. p_PS is compared against p_ET to detect incorrect predictions.
Definition 7 (Consistency condition)	The state is consistent if $P_et[i,j] \subseteq P_ps[i,j]$ for all unresolved cells. Violation signals an incorrect prediction.
Definition 8 (Prediction ID)	A unique monotonically increasing integer assigned to each prediction event, establishing total ordering for tie-breaking in cell selection.
Rank (r)	The block size n = r used to construct the grid. A rank-r puzzle has block size r, unit size $N = r^2$, and $N^2 = r^4$ cells total. Rank 3 gives the standard 9×9 puzzle. Rank 9 gives an 81×81 grid.

Table 1. Definitions used throughout the paper.

4. The CTST Method

4.1 Rules 1–4: Standard Constraint Propagation

The first four rules implement standard constraint propagation adapted for any N×N grid. Rule 3 is retained as a fast-path optimisation despite being logically subsumed by Rules 5–6; detecting naked pairs without the full ascending tally is cheaper for the common case.

- **Rule 1 — Naked Singles:** When C(i,j) = {v}, assign V(i,j) := v; remove v from all peer candidate sets.
- **Rule 2 — Hidden Singles:** If symbol s appears in exactly one unresolved cell in a unit, assign s to that cell.
- **Rule 3 — Naked Pairs (fast-path optimisation):** If two unresolved cells share identical two-element candidate sets {a,b}, remove a and b from all other cells in that unit. This is the k=2 special case of Rule 5.
- **Rule 4 — Pointing Pairs / Box-Line Reduction:** If symbol s in a row or column appears only within one block, remove s from all other unresolved cells in that block. The dual applies.

4.2 Rules 5 and 6 — The Two-Pass n-Cell-to-n-Symbol Tally

4.2.1 Motivation

Prior literature covers naked/hidden subsets only for small fixed sizes on 9×9 grids. Rules 5–6 extend this to all sizes and all N. The conceptual pass is $O(N^2)$ per unit; the implementation (including the CTS rotation sub-routine) is $O(N^4)$ per unit, making the full sweep $O(N^5)$. All complexity figures refer to implementation cost.

4.2.2 Rule 5 — Ascending Tally (Naked Subsets)

Cells are sorted in ascending order by |C(i,j)|. When s = c at depth k, the k cells form a tight naked subset. All k symbols are removed from every other unresolved cell in the unit.

Example (4×4 grid). Candidate sets {1,2}, {1,2}, {1,2,3}, {1,2,3,4}. Sorted: [{1,2},{1,2},{1,2,3},{1,2,3,4}]. Accumulate cell 1: s=2, c=1. Accumulate cell 2: s=2, c=2. Tight naked pair {1,2} detected. Symbols 1 and 2 removed from cells 3 and 4, leaving {3} and {3,4}.

4.2.3 Rule 6 — Descending Tally (Hidden Subsets)

Cells are sorted in descending order by |C(i,j)|. When s = c at depth k, the k cells form a tight hidden subset: all symbols in their union not in the tight group's symbol set are removed from those k cells.

4.2.4 Left-Side Singleton Collision Error (s − c = 0)

Lemma: If in a tight group G, two distinct symbols each appear in exactly one candidate cell and that cell is the same cell, no valid assignment exists. When s − c = 0 and any group cell c has solo(c) ≥ 2, PREDICTION ERROR is signalled immediately and rollback is initiated.

4.3 Rule 7 — Bound-Sets Filter (Symbol-First Accumulation)

4.3.1 Scope and Relationship to Prior Work

Rule 7 accumulates symbols in ascending order of cell-occurrence count. When the accumulated symbol count equals the cell count, all other candidates in those cells are removed (Hall's theorem [5]). Its detection power is theoretically subsumed by Régin [6], but it is complementary to Rules 5–6 in the current implementation: Rules 5–6 eliminate from outside tight groups (cell-first); Rule 7 eliminates from inside bound cells (symbol-first). Rule 7 fires only

at the residual fixed point of Rules 1–6 and Rule 8; even a single elimination restarts the full pipeline.

4.3.2 Algorithm

```
Initialise S_acc := {}, T_acc := {}
For each symbol s in ascending order of |cells(s)|:
```

- `S_acc := S_acc ∪ {s}`
- `T_acc := T_acc ∪ cells(s)`
- `If |S_acc| = |T_acc|:  eliminate all σ ∉ S_acc from every c ∈ T_acc`
- `If |S_acc| > |T_acc|:  signal missing-symbol error`
- `If |S_acc| < |T_acc|:  continue accumulating`

Lemma: If |S| symbols collectively appear in exactly |S| cells, all other candidates in those cells can be eliminated (Hall's theorem [5]).

4.4 Rule 8 — Near-Tight Subset Analysis

4.4.1 Gap in the Surveyed Literature

Existing alldifferent filtering algorithms primarily target tight subsets (s − c = 0), enforcing consistency via Hall's condition. Near-tight configurations (s − c = n for positive integer n) are typically handled implicitly within full generalised arc-consistency (GAC) propagation rather than through a dedicated lightweight procedure.

To our knowledge, existing literature does not isolate the near-tight regime as a distinct operational case with an explicit, lightweight procedure for any n. We do not claim new deductive power beyond GAC; the contribution lies in providing a direct operationalisation of this regime — for all n as a general theorem, implemented for n = 1 and n = 2.

4.4.2 Overview

Sections §4.4.2–§4.4.5 describe Rule 8 for the n = 1 case (s − c = 1) in detail. Section §4.4.6 covers n = 2. The general theorem for all n is presented in §4.4.8.

Phase 1 — Resolve. If exactly one left-side cell c* has solo = 2 with singletons {s1, s2}, the shared symbols G_sym \ {s1, s2} are removed from all right-side cells of the unit.

Phase 2 — Error. If any left-side cell has solo > 2, the configuration is a logical impossibility. PREDICTION ERROR is signalled immediately and rollback is initiated.

4.4.3 Mechanism

Case	Condition	Outcome	Action
Resolve	Exactly one left-side cell c* with solo = 2; singletons = {s1, s2}	FILTER	Remove G_sym \ {s1, s2} from all right-side cells
Error	Any left-side cell with solo > 2	ERROR	Rollback immediately
Skip	No left-side cell with solo ≥ 2	SKIP	Insufficient information — continue

Table 2. Rule 8 case table for s − c = 1.

4.4.4 Worked Example — Resolve Phase

Setup. A unit is partitioned into a left group G of 5 cells and a right group of 5 remaining unit cells.

```
Left group (G):  {1,2,3,4,5} · {1,2,3,6} · {1,2,3,6} · {1,2,3} · {1,2,3}
Right group:     {5,6} · {1,2,7,8} · {1,5,6,7,8} · {7,8,4} · {7,4,9}
```

Step 1. G_sym = {1,2,3,4,5,6}, s = 6, c = 5, s−c = 1 → Near-tight. Rule 8 fires.

Step 2. Symbol 4: cell 1 only (singleton). Symbol 5: cell 1 only (singleton). Cell 1 has solo = 2, singletons = {s1 = 4, s2 = 5}. Resolve phase fires. Cell 1 is c*.

Step 3. Right-side filtering. Shared symbols = G_sym \ {s1, s2} = {1,2,3,4,5,6} \ {4,5} = {1,2,3,6}. These are removed from all right-side cells: {5,6} → {5}; {1,2,7,8} → {7,8}; {1,5,6,7,8} → {5,7,8}; {7,8,4} → unchanged; {7,4,9} → unchanged. The pipeline restarts from Rule 1.

4.4.5 Worked Example — Error Phase

Setup. A unit contains a left group G of 5 cells.

```
Left group (G):  {1,2,3,4,5,7} · {1,2,3} · {1,2,3} · {1,2,3} · {1,2,3}
```

Step 1. G_sym = {1,2,3,4,5,7}. s = 6, c = 5, s − c = 1 → Near-tight. Rule 8 fires.

Step 2 — Left-side solo count. Symbols 1, 2, 3 appear in all five cells (shared). Symbols 4, 5, 7 each appear in cell 1 only (singleton). Cell 1 has solo = 3.

Symbol	Appears in cells	Count	Classification
1	1, 2, 3, 4, 5	5	Shared
2	1, 2, 3, 4, 5	5	Shared
3	1, 2, 3, 4, 5	5	Shared
4	Cell 1 only	1	Singleton in cell 1
5	Cell 1 only	1	Singleton in cell 1
7	Cell 1 only	1	Singleton in cell 1

Table 2a. Symbol solo-count for the Error phase example. Cell 1 has solo = 3.

Step 3 — Error phase fires immediately. solo(cell 1) = 3 > 2. Error phase fires immediately. The group is near-tight (s − c = 1): exactly one symbol must be placed outside the group. However, cell 1 holds three symbols — 4, 5, and 7 — each appearing nowhere else in G. The configuration is a logical impossibility.

Step 4 — Outcome. PREDICTION ERROR is signalled immediately. The solver traces triggerLv records, backjumps to the originating prediction, and initiates rollback. The right group is never examined.

4.4.6 Extension to s − c = 2

When s − c = 2, exactly two symbols must leave G as surplus. The Resolve condition requires exactly two cells in G each carrying exactly two singleton-occurrence symbols: cell A holds singletons {s1, s2} and cell B holds singletons {s3, s4}, with every other G-cell holding no singletons. If more than two such cells exist, a prediction error is signalled immediately and rollback is initiated.

The shared symbols G_sym \ {s1, s2, s3, s4} are therefore removed from both cell A and cell B, reducing each to a naked pair. Filtering also applies to right-side cells: shared symbols cannot leave G and are removed from all cells outside G in the unit.

4.4.7 Worked Example — s − c = 2 Resolve Phase

Group	Cell	Candidates
Left (G)	Cell 1	{1, 2, 3, 4, 5, 6}
	Cell 2	{1, 2, 3, 4, 8, 9}
	Cell 3	{1, 2, 3, 4}
	Cell 4	{1, 2, 3, 4}
	Cell 5	{1, 2, 3, 4}
	Cell 6	{1, 2, 3, 4}
Right	Cell 7	{1, 4, 5, 7}
	Cell 8	{1, 2, 5, 8}
	Cell 9	{1, 3, 7, 9}

Table 2b. s − c = 2 worked example setup.

Step 1. G_sym = {1,2,3,4,5,6,8,9}. s = 8, c = 6, s − c = 2 → Rule 8 fires.

Step 2 — Solo count. Cell 1: solo = 2, singletons {s1=5, s2=6}. Cell 2: solo = 2, singletons {s3=8, s4=9}. All other cells: solo = 0. Exactly two sets → Resolve fires.

Step 3 — Left-group filtering. Shared = {1,2,3,4}. Cell 1: {1,2,3,4,5,6} → {5,6} (naked pair). Cell 2: {1,2,3,4,8,9} → {8,9} (naked pair).

Step 4 — Right-side filtering. Remove {1,2,3,4} from all right-side cells. The pipeline restarts from Rule 1.

4.4.8 Generalisation — Rule 8 for s − c = n

The near-tight analysis generalises to any surplus degree n. For a group G with s − c = n, the Resolve condition is: exactly n cells in G each carry exactly two singleton-occurrence symbols, and no other G-cell carries singletons. Cell A_i holds singletons {s_{2i−1}, s_{2i}} for i = 1, …, n. All remaining symbols in G_sym are shared.

Filtering. Shared symbols G_sym \ {s1, s2, …, s_{2n}} are removed from each of the n singleton cells, reducing each to a naked pair. Shared symbols are also removed from all right-side cells of the unit.

Error condition. If more than n cells in G carry singletons, a prediction error is signalled immediately and rollback is initiated.

Proof. Exactly n surplus symbols must leave G. Each singleton cell A_i takes one of its two singletons and contributes the other as surplus: n cells, one surplus each, giving n surpluses in total. A shared symbol cannot be surplus — if it left G, the 2n singletons across the n cells would require n+1 or more surplus slots, contradicting s − c = n. Shared symbols therefore cannot leave G and are safely removed from all singleton cells and all right-side cells.

4.5 CTST Pipeline Order

Stage	Rule	Regime	Direction	Output	Notes

1	Rule 1 — Naked Singles	Single cell	Cell → Value	Cell resolution	
2	Rule 2 — Hidden Singles	Single symbol per unit	Symbol → Cell	Cell resolution	
3	Rule 3 — Naked Pairs (fast-path)	Two-cell tight	Cell → Symbol	Candidate elimination	k=2 special case of Rule 5
4	Rule 4 — Pointing Pairs	Row/col × block intersection	Symbol → Block	Candidate elimination	
5	Rule 5 — Ascending Tally	Tight (s−c=0), all sizes	Cell → Symbol	Candidate elimination; collision error	
6	Rule 6 — Descending Tally	Tight (s−c=0), all sizes	Cell → Symbol	Candidate elimination	
7	Rule 7 — Bound-Sets Filter	Tight (|S|=|T|), symbol-first	Symbol → Cell	Candidate elimination (inside bound cells)	
8	Rule 8 — Near-Tight Analysis	Near-tight (s−c=1, s−c=2)	Cell → Symbol (solo)	Filter / error / key cell	

Table 3. CTST pipeline order. When any rule produces at least one candidate elimination, control returns immediately to Stage 1.

5. Prediction of Key Cell Symbols (Non-Deterministic Phase)

5.1 Cell Selection Strategy

The key cell (Definition 6) is the unresolved cell with the smallest candidate set size. Ties are broken by prediction ID (Definition 8). No restriction is placed on candidate set size: if the smallest available set has k candidates (for any k ≥ 2), the solver predicts one of those k values and propagates.

The empirical results in §8 show that the deterministic phase (Rules 1–8) resolves the majority of cells before prediction is needed, keeping typical candidate set sizes small at the moment of prediction.

5.2 Prediction Error Rate

Across the tested samples, fewer than 50% of prediction steps require rollback (Tables A2–A3). The rate is stable at 36–44% across ranks. For context, a naive random predictor yields a 50% rollback rate for 2-candidate cells and 67% for 3-candidate cells; the observed rate therefore represents meaningful improvement over random for 3-candidate predictions, but only marginal improvement for 2-candidate predictions. Whether the cross-rank stability reflects a structural property of near-minimal Sudoku puzzles or a solver artefact is identified as future work (see §9.3).

6. Error Detection and Rectification

6.1 Detection Phase

Errors are identified through inconsistencies between the expected propagation state P_et and the observed propagation state P_ps. Rule 8 Phase 2 (Error) detects impossible near-tight configurations: for s − c = 1, any cell with solo > 2 triggers immediate rollback; for s − c = 2, more than two singleton-carrying cells triggers immediate rollback. The Rules 5–6 left-side singleton collision error detects internal tight-group contradictions.

6.2 Error Classification

Error Type	Description	Detection Trigger	Resolution Action
Cells Conflict	Same symbol assigned as singleton to two cells in the same unit	During symbol insertion	Rollback via triggerLv; replace incorrect symbol with next candidate
Symbols Conflict	Two distinct symbols both resolve to the same cell	During hidden-single detection (Rule 2)	Rollback to originating prediction; causal rectification
Empty Cell / Empty Options	A cell has no symbol and empty options table entry	Every options update	Rollback; treat as consistency violation (Definition 7)
Data Integrity Violation	Options table inconsistent with puzzle board	Post-update integrity assertion	Rollback and re-synchronise to last consistent checkpoint
Missing Symbol	A symbol required in a unit has no candidate cell	Every options update	Rollback; causal rectification with next available candidate
s−c Inconsistency	A structural impossibility within a tight or near-tight group. Two sub-cases: (i) Tight (s−c=0): a cell within the tight group holds solo(c) ≥ 2 — two or more symbols each appearing in exactly one group cell, yet only one cell can hold a value, making assignment impossible (Left-Side Singleton Collision). (ii) Near-tight (s−c=n): more than n cells in G each carry exactly two singleton-occurrence symbols — the available surplus slots (n) cannot accommodate all singletons, making a valid assignment impossible.	During Rules 5–6 (tight group tally) and Rule 8 (near-tight phase), on any unit update	Rollback via triggerLv; backjump directly to the originating prediction; initiate causal rectification with the next available candidate
Error Loop	Same inconsistency recurs after rollback	Rectification counter exceeds candidate set size	Backjump to parent predicted cell; mark both candidates invalid; resume from parent

Table 4. Error classification. All seven error types recognised by the CTST solver.

6.3 Rectification Phase

The rectification phase integrates conflict-directed backjumping from CSP theory with Sudoku-specific propagation. The triggerLv mechanism operationalises ideas from Dechter's framework [13], enabling direct backjumping to the root cause of inconsistencies rather than chronological rollback.

- **Root Cause Identification:** solver traces triggerLv records and backjumps directly to the originating prediction, following the conflict-directed backjumping paradigm of Dechter [13].
- **Hint Extraction During Rollback:** predicted symbols from cells assigned after the root-causing cell are recorded as hints for reuse.
- **State Restoration:** P_et and P_ps are restored to the state corresponding to the root-causing cell.
- **Causal Rectification:** the incorrect symbol is replaced with the next available candidate.
- **Hint-Driven Prediction:** stored hints are reused before new predictions are made.
- **Consistency Restoration:** Rules 1–8 are reapplied until a new fixed point is reached.

7. Complexity Analysis

7.1 Notation

N denotes the grid side-length ($N = n^2$, n = block size). The grid has 3N units, N^2 cells, and N candidate symbols per cell, giving a state space of N^3 (cell, symbol) pairs. K is the total number of prediction steps; R is the number of convergence passes in the deterministic phase. b denotes the branching factor at a given prediction step; in the worst case $b \leq N$.

7.2 Space Complexity

$O(N^3)$. The two 3D candidate tables p_ET and p_PS each have dimensions $N^2 \times (N+4)$. All working buffers are $O(N^2)$ and do not change the leading term.

7.3 Per-Rule Time Complexity

Important distinction: the conceptual tally pass of Rules 5–6 is $O(N^2)$ per unit; the current implementation, including the CTS rotation sub-routine, is $O(N^4)$ per unit, making the full sweep $O(N^5)$. All complexity claims refer to the implementation cost.

Rule	Per-unit	Pass total (3N units)	Dominant function
Init	$O(N)$	$O(N^3)$	build_options_table
Rule 1 — Naked Singles	$O(N^2)$	$O(N^3)$ amort.	initialise_insert_symbol_into_puzzle_table
Rule 2 — Hidden Singles	$O(N^2)$	$O(N^3)$	find_hidden_symbol
Rule 3 — Naked Pairs	$O(N^2)$	$O(N^3)$	find_twin_set
Rule 4 — Pointing Pairs	$O(N^2 \cdot \sqrt{N})$	$O(N^3 \sqrt{N})$	filter_block_symbol
Rules 5–6 — Tally	$O(N^4)$	$O(N^5)$	find_n_symbol_n_cell_tally_dsc / asc
Rule 7 — Bound-Sets	$O(N^3)$	$O(N^4)$	remove_invalid_options_from_bound_cells
Rule 8 — Near-tight	$O(N^4)$	$O(N^5)$	filter_symbol_sets_group_method

Table 5. Per-rule time complexity. Implementation-derived.

7.4 Full Deterministic Phase

- **Total candidate eliminations:** $O(N^3)$. Each of the $N^2 \times N$ candidate slots is removed at most once — a hard monotone upper bound independent of R.
- **Per convergence pass:** $O(N^5)$, dominated by Rules 5–6 and Rule 8.
- **Total:** $O(R \cdot N^5)$. A tight bound on R is an open problem; empirically R is small.

7.5 Prediction Phase

- **Per prediction step:** $O(N^5)$.
- **Full solve:** $O(K \cdot N^5)$. K is bounded by N^2 in the worst case, giving an overall bound of $O(N^7)$.
- **Empirical scaling:** t ~ K^{3.17} ($R^2 = 0.97$). Derived from experimental data in which K was varied from 1 to 14.

7.6 Complexity Summary

Phase	Complexity	Status
Initialisation	$O(N^3)$	Firm
Single convergence pass — Rules 1–4	$O(N^3\sqrt{N})$	Impl.-derived
Single convergence pass — Rules 5–6 and Rule 8 (dominant)	$O(N^5)$	Impl.-derived
Single convergence pass — Rule 7	$O(N^4)$	Firm
Total candidate eliminations	$O(N^3)$	Hard upper bound
Full deterministic phase	$O(R \cdot N^5)$	R tight bound open
Per prediction step	$O(N^5)$	Impl.-derived
Full solve (K prediction steps)	$O(K \cdot N^5)$	$K \leq N^2$
Space	$O(N^3)$	Two 3D tables
Overall worst case	$O(N^7)$	$K = N^2$; impl.-derived

Table 6. Complexity summary.

7.7 Gap Between Worst-Case Complexity and Empirical Performance

The $O(N^7)$ bound and observed runtimes differ by many orders of magnitude. A rank-18 grid has a theoretical ceiling of $324^7 \approx 4 \times 10^{17}$ operations; the maximum-key-cell instance at rank 18 completes in under 5 hours, the zero-key-cell instance in 9 seconds. After $O(N^3)$ cumulative eliminations, the puzzle is either fully resolved or at a fixed point. Even if R were large, each pass applying $O(N^5)$ work must make at least one elimination to avoid being a null pass, so $R \leq O(N^3)$ in the absolute worst case. In practice R is observed to be a small constant (typically 2–5 passes).

7.7.1 Summary of the Gap

Factor	Theoretical assumption	Empirical reality	Effect on cost

Prediction count K	$K = N^2$ (worst case)	$K \ll N^2$; median K = 2 at rank 3	Reduces $O(N^7)$ to $O(K \cdot N^5)$ with K small
Branching factor b	b = N (full candidate set)	b typically 2–3 at moment of prediction	Reduces b^K exponential to manageable levels
Monotone elimination budget	R unbounded	Total eliminations $\leq N^3$; R small constant empirically	Caps total work at $O(N^3)$ eliminations
Rules 5–6 trigger rate	Full tally on all 3N units every pass	Most units resolved by Rules 1–4 before tally fires	$O(N^5)$ per-pass cost is an envelope, not typical
Error detection timing	Full $O(N^5)$ pass before inconsistency detected	Rule 8 / collision errors fire immediately on unit update	Prunes bad branches at minimal cost
First-trigger restart	All eight rules execute on full unit state every pass	Any firing rule restarts from Rule 1	Per-unit cost of Rules 5–8 is small fraction of envelope

Table 7. Sources of the gap between the $O(N^7)$ worst-case bound and observed runtimes.

8. Experimental Results

Scope. Tests span ranks 3 through 18 for maximum-key-cell instances and ranks 3 through 19 for minimum-key-cell instances. Ranks 19–20 maximum-key-cell runs were not attempted: the rank 18 maximum-key-cell instance required 4 hours 52 minutes, placing higher ranks outside current computational limits. Prediction-phase benchmarks use reference-solution validation rather than the full autonomous rectification mechanism, isolating propagation behaviour across the tested instances.

8.1 Royle Corpus Results (Rank 3, 9×9)

The Royle corpus is used to measure how many standard 9×9 puzzles the deterministic phase alone can solve — without any prediction. Of the 49,158 puzzles tested, 84.7% were resolved entirely by Rules 1–8, requiring no prediction step at all.

Grid	Source	Corpus Size	Tested	Solved by Filters Alone	Required Prediction
9×9	Royle [7]	49,158	49,158	41,650 (84.7%)	7,508 (15.3%)

Table 8. Royle corpus results: proportion of 9×9 puzzles solved by the deterministic phase alone.

8.2 CTST Performance Summary Across Grid Sizes (max key cells)

Grid size	9×9	16×16	25×25	36×36	49×49	64×64
Samples	50	50	50	50	20	20
Avg. solve time (s)	0.00	0.00	0.00	0.01	0.03	0.09

Table 9. CTST average solve time across grid sizes.

8.3 Effect of Rules 7 and 8: Ablation Study

To evaluate the contribution of Rules 7 (Bound-Sets Filter) and 8 (Near-Tight Analysis), an ablation study was conducted across four grid sizes (16×16, 25×25, 36×36, and 49×49). Each configuration was executed on identical puzzle instances under two conditions: (i) the full CTST pipeline (Rules 1–8, labelled "Full"), and (ii) a reduced configuration with Rules 7 and 8 disabled (labelled "−R7/8").

Grid	Config	M1 Pred.	M2 Single.	M3 Hidden	M4 N.Pair	M5 Block	M6 Subset	M7 T.G.	M8 N.Tight	M9 Bnd.Set
16×16	Full	33,214	40,936	13,806	36,498	8,145	19,806	12,865	362	6,278
	−R7/8	33,531	40,475	13,929	54,129	7,293	19,019	1,742	—	—
25×25	Full	82,346	89,019	31,644	111,148	20,168	53,305	41,720	867	18,326
	−R7/8	82,393	88,091	32,525	152,765	18,952	56,773	10,466	—	—
36×36	Full	167,683	146,418	66,053	292,695	41,533	113,686	75,820	1,335	29,111
	−R7/8	167,205	144,063	68,886	229,315	45,613	104,899	22,520	—	—
49×49	Full	53,854	52,091	24,852	88,364	14,773	38,317	32,428	807	11,619
	−R7/8	53,483	51,681	25,633	113,884	13,564	42,582	10,450	—	—

Table 10. Ablation study: cumulative rule-invocation counts, full pipeline vs. Rules 7 and 8 disabled.

8.3.1 Impact on Deterministic Computation

Early-stage metrics (M1–M3) remain largely stable across both configurations at all grid sizes, confirming that basic constraint propagation (Rules 1–4) operates independently of higher-order filtering. In contrast, M4 (Naked Pair) increases substantially when Rules 7 and 8 are removed: by approximately 29% in the 49×49 case (88,364 → 113,884), with similar trends across smaller grids.

8.3.2 Impact on Structured Propagation

Rules 7 and 8 significantly increase structured propagation events (M7), indicating that tight-group reasoning is triggered more frequently in the full configuration. This reflects improved structural awareness of candidate distributions rather than increased computational overhead.

8.3.3 Prediction Behaviour and Trade-off

M1 (Prediction) is slightly higher in the full configuration across all grid sizes. This reflects a structural trade-off: Rules 7 and 8 prune candidate structures aggressively and expose ambiguity earlier, causing the solver to reach deterministic fixed points sooner and initiate prediction earlier.

8.3.4 Scaling Behaviour

The divergence between the two configurations grows with grid size, particularly in M4. This confirms that Rules 7 and 8 are increasingly important for scalability: their structural pruning prevents the quadratic growth in naked-pair invocations that would otherwise dominate runtime at larger ranks.

8.3.5 Summary

Although Rules 7 and 8 account for a small fraction of direct candidate eliminations (M8, M9), their structural impact is substantial. Their presence reduces high-order deterministic computation (M4↓, M6↓), enables tight-group inference (M7↑), and improves scalability across larger grids.

9. Discussion

9.1 Relation to Minimal Clue Results and Extremal Instances

The minimum number of clues required for a uniquely solvable Sudoku is known to be 17 for the standard 9×9 case [16]. For higher-rank generalisations, however, no such minimum is known, and exhaustive determination is widely considered computationally infeasible. Across all tested instances — zero-key-cell up to rank 20, minimum-key-cell up to rank 19, maximum-key-cell up to rank 18 — all puzzles were solved without any halt due to candidate-count issues.

9.2 Parallel Error Detection Coverage

Rules 5–6 ($s - c = 0$) and Rule 8 ($s - c = n$, implemented for $n = 1$ and $n = 2$, general theorem for all n) provide non-redundant coverage from opposite elimination directions; Rule 7 adds symbol-first inside-cell elimination. Together they cover tight and near-tight regimes. The $s - c < 0$ case is caught by the standard consistency check (Definition 7).

9.3 Future Work Priorities

- **Solver comparison (Version 2).** A controlled runtime comparison between CTST and DLX [3] and a competitive SAT solver across ranks 3–20 will be presented in Version 2 of this preprint, alongside fully independent solver data.
- **Rule 8 extension to $s - c \geq 3$.** The $s - c = 1$ and $s - c = 2$ regimes are both implemented. Extension follows the same structural pattern but such configurations are rare in practice.
- **Empirical scaling — formal analysis.** The $t \sim K^{3.17}$ relationship ($R^2 = 0.97$), derived from experimental data across K = 1–14, invites a formal derivation.
- **Prediction error rate analysis.** Whether alternative prediction ordering heuristics reduce the rollback rate. Across all tested ranks, the rollback rate clusters consistently near 40–44%.
- **Large-candidate-set prediction heuristics.** Developing smarter value-ordering heuristics (e.g., least-constraining value, domain-specific frequency analysis).
- **Literature coverage.** An exhaustive search found no prior work presenting a dedicated lightweight algorithm for the near-tight regime ($s - c = n$). The novelty claim for Rule 8 is therefore a contribution of explicit operationalisation.

10. Conclusion

CTST is a practical constraint-propagation solver for generalised N×N Sudoku. Three main contributions are presented: (1) Rules 5–6 detect tight subsets of any size via a two-pass tally ($O(N^4)$ per unit), unifying naked and hidden subset detection; (2) Rule 8 provides a general near-tight filtering theorem for any surplus degree n ($s - c = n$): when exactly n cells in G each carry exactly two singleton-occurrence symbols, shared symbols are eliminated from those n

cells and all right-side cells — implemented for n = 1 and n = 2, with the general theorem holding for all n; (3) Rule 7 complements Rules 5–6 by eliminating candidates from inside bound cells via symbol-first accumulation.

On the Royle corpus of 49,158 nine-by-nine puzzles, the deterministic phase alone solved 84.7% without any prediction. The ablation study shows Rules 7 and 8 reduce naked-pair invocations by up to 29% and triple tight-group detections at larger ranks.

Open problems: Rule 8 implementation extension to s − c ≥ 3 (general theorem proven in §4.4.8, implementation covers n = 1 and n = 2); ranks 19–20 maximum-key-cell results (pending sufficient compute); formal derivation of the empirical t ~ K^{3.17} scaling law; large-candidate-set prediction heuristics.

References

[1] T. Yato and T. Seta, “Complexity and completeness of finding another solution,” IEICE Trans. Fundamentals, vol. E86-A, no. 5, pp. 1052–1060, 2003.

[2] P. Norvig, “Solving every Sudoku puzzle,” 2006. [Online]. Available: https://norvig.com/sudoku.html

[3] D. E. Knuth, “Dancing links,” in Millennial Perspectives in Computer Science. Palgrave, 2000, pp. 187–214.

[4] J. P. Marques-Silva, I. Lynce, and S. Malik, “Conflict-driven clause learning SAT solvers,” in Handbook of Satisfiability. IOS Press, 2009, pp. 131–153.

[5] P. Hall, “On representatives of subsets,” Journal of the London Mathematical Society, vol. 10, pp. 26–30, 1935.

[6] J.-C. Régin, “A filtering algorithm for constraints of difference in CSPs,” in Proc. AAAI-94, pp. 362–367, 1994.

[7] G. Royle, “Minimum Sudoku — 17-clue puzzle corpus,” University of Western Australia.

[8] H. Simonis, “Sudoku as a constraint problem,” in Proc. 4th Int. Workshop on Modelling and Reformulating CSPs, 2005, pp. 13–27.

[9] M. Leconte, “A bounds-based reduction scheme for constraints of difference,” in Proc. Constraint-96, pp. 19–28, 1996.

[10] W. J. van Hoeve, “The alldifferent constraint: A survey,” in Proc. 6th Annual Workshop of the ERCIM Working Group on Constraints, Prague, 2001.

[11] J.-F. Puget, “A fast algorithm for the bound consistency of alldiff constraints,” in Proc. AAAI-98, pp. 359–366, 1998.

[12] A. Lopez-Ortiz, C.-G. Quimper, J. Tromp, and P. van Beek, “A fast and simple algorithm for bounds consistency of the alldifferent constraint,” in Proc. 18th IJCAI, pp. 245–250, 2003.

[13] R. Dechter, Constraint Processing. San Francisco, CA: Morgan Kaufmann, 2003.

[14] I. Lynce and J. Ouaknine, “Sudoku as a SAT problem,” in Proc. 9th Int. Symp. on AI and Mathematics, Fort Lauderdale, Florida, 2006.

[15] I. P. Gent, C. Jefferson, I. Miguel, and P. Nightingale, “Generalised arc consistency for the alldifferent constraint: Exploiting strongly connected components,” Journal of Artificial Intelligence Research, 2008.

[16] G. McGuire, B. Tugemann, and G. Civario, “There is no 16-clue Sudoku: Solving the Sudoku minimum number of clues problem,” Experimental Mathematics, vol. 23, no. 2, pp. 190–217, 2014.

Appendix A — Detailed Per-Rank Experimental Results

This appendix contains the full per-rank breakdowns referenced in §8.2.

A1. Zero-Key-Cell Results Across Ranks 9–20

Rank	Grid Size	Init. Filled	To Fill	Filled	Key Cells	Pred. Errors	Elapsed
9	81×81	4,310	2,251	2,251	0	0	00:00:00
10	100×100	6,747	3,253	3,253	0	0	00:00:01
11	121×121	10,101	4,540	4,540	0	0	00:00:01
12	144×144	14,664	6,072	6,072	0	0	00:00:01
13	169×169	20,752	7,809	7,809	0	0	00:00:01
14	196×196	28,077	10,339	10,339	0	0	00:00:02
15	225×225	37,359	13,266	13,266	0	0	00:00:02
16	256×256	49,844	15,692	15,692	0	0	00:00:02
17	289×289	63,623	19,898	19,898	0	0	00:00:11
18	324×324	81,212	23,764	23,764	0	0	00:00:09
19	361×361	101,531	28,790	28,790	0	0	00:00:23
20	400×400	124,974	35,026	35,026	0	0	00:00:52

Table A1. Zero-key-cell results, ranks 9–20.

A2. Minimum-Key-Cell Results Across Ranks 9–19

Rank	Grid Size	Init. Filled	To Fill	Filled	Key Cells	Pred. Errors	Elapsed
9	81×81	3,995	2,566	2,251	22	8 (36.4%)	00:00:01
10	100×100	6,356	3,644	3,253	155	57 (36.8%)	00:00:04
11	121×121	9,495	5,146	4,540	101	38 (37.6%)	00:00:08
12	144×144	13,787	6,949	6,072	251	111 (44.2%)	00:00:28
13	169×169	19,328	9,233	7,809	291	122 (41.9%)	00:00:53
14	196×196	26,618	11,798	10,339	474	195 (41.1%)	00:02:03

15	225×225	35,624	15,001	13,266	388	150 (38.7%)	00:05:21
16	256×256	46,641	18,895	15,692	759	307 (40.5%)	00:12:34
17	289×289	60,733	22,788	19,898	945	388 (41.1%)	00:16:39
18	324×324	77,445	27,531	23,764	939	382 (40.7%)	00:28:56
19	361×361	95,983	34,338	28,790	1,513	638 (42.2%)	00:57:28

Table A2. Minimum-key-cell results, ranks 9–19.

A3. Maximum-Key-Cell Results Across Ranks 9–18

Rank	Grid Size	Init. Filled	To Fill	Filled	Key Cells	Pred. Errors	Elapsed
9	81×81	3,500	3,061	1,728	1,333	540 (40.5%)	00:00:10
10	100×100	5,723	4,277	2,539	1,738	719 (41.4%)	00:00:21
11	121×121	8,648	5,993	3,214	2,779	1,123 (40.4%)	00:00:57
12	144×144	12,824	7,912	4,597	3,315	1,351 (40.8%)	00:02:07
13	169×169	18,175	10,386	5,877	4,500	1,841 (40.9%)	00:04:49
14	196×196	25,128	13,288	7,676	5,612	2,192 (39.1%)	00:09:44
15	225×225	33,899	16,726	9,082	6,744	2,748 (40.8%)	00:18:52
16	256×256	45,317	20,219	12,011	8,208	3,061 (37.3%)	00:35:24
17	289×289	58,804	24,717	14,938	9,779	3,657 (37.4%)	01:03:10
18	324×324	73,999	30,977	18,118	12,859	5,274 (41.0%)	04:52:14

Table A3. Maximum-key-cell results, ranks 9–18.

Appendix B — Derivation of K Values from Experimental Data

B1. What K Measures

K is the total number of prediction steps (key cell selections) made by the solver on a given puzzle instance. Each prediction step consists of: (1) selecting the unresolved cell with the smallest candidate set, (2) assigning one candidate value, and (3) propagating Rules 1–8 until a new fixed point is reached. If propagation produces an inconsistency, that step is counted as a prediction error and the solver rolls back. Both successful and failed prediction steps are

counted in K. K is therefore an empirical measure of how much the deterministic phase (Rules 1–8) leaves unresolved: a larger K indicates that constraint propagation alone was insufficient to close more cells, requiring more branching.

B2. K Values Across the Three Experimental Regimes

B2.1 Zero-Key-Cell Regime (Table A1)

In the zero-key-cell regime, K = 0 for every instance across ranks 9–20. This means Rules 1–8 alone resolved all remaining cells without any prediction step. The solver reached a complete fixed point deterministically. The elapsed times in Table A1 (ranging from under 1 second at rank 9 to 52 seconds at rank 20) reflect purely the cost of the deterministic phase, making this regime the cleanest benchmark for Rules 1–8 in isolation.

B2.2 Minimum-Key-Cell Regime (Table A2)

In the minimum-key-cell regime, K represents the smallest number of prediction steps observed for a solvable instance at each rank. The K values and their relationship to the cells remaining to fill are shown in Table B1.

Rank	Grid	To Fill	K (min)	K / To Fill %	Pred. Errors	Error Rate %
9	81×81	2,566	22	0.9%	8	36.4%
10	100×100	3,644	155	4.3%	57	36.8%
11	121×121	5,146	101	2.0%	38	37.6%
12	144×144	6,949	251	3.6%	111	44.2%
13	169×169	9,233	291	3.2%	122	41.9%
14	196×196	11,798	474	4.0%	195	41.1%
15	225×225	15,001	388	2.6%	150	38.7%
16	256×256	18,895	759	4.0%	307	40.5%
17	289×289	22,788	945	4.1%	388	41.1%
18	324×324	27,531	939	3.4%	382	40.7%
19	361×361	34,338	1,513	4.4%	638	42.2%

Table B1. Minimum-key-cell K values derived from Table A2, with K as a percentage of cells remaining to fill.

Two observations follow directly from Table B1. First, K in the minimum-key-cell regime is consistently small relative to the number of cells to fill: across all ranks, K represents only 0.9–4.4% of the cells that must be resolved. This confirms that the deterministic phase (Rules 1–8) resolves the vast majority of cells — between 95.6% and 99.1% — without any prediction. Second, the prediction error rate is stable at 36–44% across all ranks, independent of grid size. This means that on average a prediction error occurs approximately once every 2.3–2.8 prediction steps regardless of rank, indicating a consistent structural property of the puzzle instances rather than solver-specific behaviour.

B2.3 Maximum-Key-Cell Regime (Table A3)

In the maximum-key-cell regime, K represents the largest number of prediction steps observed for a solvable instance at each rank. The K values and their relationship to cells to fill are shown in Table B2.

Rank	Grid	To Fill	K (max)	K / To Fill %	Pred. Errors	Error Rate %
9	81×81	3,061	1,333	43.5%	540	40.5%
10	100×100	4,277	1,738	40.6%	719	41.4%
11	121×121	5,993	2,779	46.4%	1,123	40.4%
12	144×144	7,912	3,315	41.9%	1,351	40.8%
13	169×169	10,386	4,500	43.3%	1,841	40.9%
14	196×196	13,288	5,612	42.2%	2,192	39.1%
15	225×225	16,726	6,744	40.3%	2,748	40.7%
16	256×256	20,219	8,208	40.6%	3,061	37.3%
17	289×289	24,717	9,779	39.6%	3,657	37.4%
18	324×324	30,977	12,859	41.5%	5,274	41.0%

Table B2. Maximum-key-cell K values derived from Table A3, with K as a percentage of cells remaining to fill.

Three structural observations follow from Table B2. First, K in the maximum-key-cell regime is approximately 40–46% of cells to fill across all ranks. This proportion is remarkably stable: despite grid size growing from 81×81 (rank 9) to 324×324 (rank 18), the fraction of cells requiring prediction varies only between 39.6% and 46.4%. This suggests that the maximum-key-cell instances share a common structural property across ranks — approximately 40% of cells cannot be resolved deterministically by Rules 1–8, regardless of grid size.

Second, each prediction step resolves on average approximately 2.3–2.5 cells per step in the maximum-key-cell regime. Since each prediction step assigns exactly one value, the remainder are resolved by the deterministic cascade that follows each prediction. This means that even in the hardest instances, the propagation cascade after each prediction resolves roughly 1.3–1.5 additional cells deterministically before the next prediction is needed.

Third, the prediction error rate remains in the 37–41% band for the maximum-key-cell regime, consistent with the minimum-key-cell regime (36–44%). The error rate is therefore independent of both the difficulty regime (min vs max key-cell) and the rank. This cross-regime and cross-rank stability is a structural property of near-minimal Sudoku instances and is identified as a direction for formal analysis in future work (§9.3).

B3. How the K = 1–14 Range for the Empirical Scaling Law Was Obtained

The empirical scaling law $t \sim K^{3.17}$ ($R^2 = 0.97$, §7.5) was derived from instances at rank 3 (standard 9×9 puzzles) drawn from the Royle corpus. Of the 49,158 Royle puzzles tested, 7,508 (15.3%) required prediction. Among these, individual puzzle K values range from 1 (a single prediction step suffices after the deterministic phase) up to 14 (the hardest instances in the corpus require 14 sequential prediction steps). The solver's elapsed time was recorded for each such puzzle and plotted against K. Fitting a power law $t = aK^b$ to this data yielded an exponent $b = 3.17$ with $R^2 = 0.97$.

The K = 1–14 range is therefore not a design choice or a truncation: it is the natural empirical distribution of K values in the 9×9 Royle prediction-requiring subset. No 9×9 puzzle in the corpus required more than 14 prediction steps under CTST. The appendix tables (A2 and A3) report single hard instances at ranks 9–19/18, where K is orders of magnitude larger; these instances are outside the rank-3 scaling law's domain and are reported separately as scalability benchmarks rather than inputs to the power-law fit.

The relationship between the rank-3 empirical law and behaviour at higher ranks is an open problem identified in §9.3. Whether the $t \sim K^{3.17}$ exponent holds as rank and K grow, or whether the exponent changes with rank, requires formal derivation and is deferred to future work.

B4. Summary of K Across All Regimes

Regime	K Range (ranks 9–19)	K as % of Cells to Fill	Prediction Error Rate
Zero-key-cell (Table A1)	K = 0 (all ranks)	0% — fully deterministic	N/A (no predictions)
Minimum-key-cell (Table A2)	22 – 1,513	0.9% – 4.4%	36–44%
Maximum-key-cell (Table A3)	1,333 – 12,859	39.6% – 46.4%	37–41%
Scaling law source (rank 3, Royle)	1 – 14	Small (9×9 instances)	Consistent with above

Table B3. Summary of K values and prediction error rates across all experimental regimes.

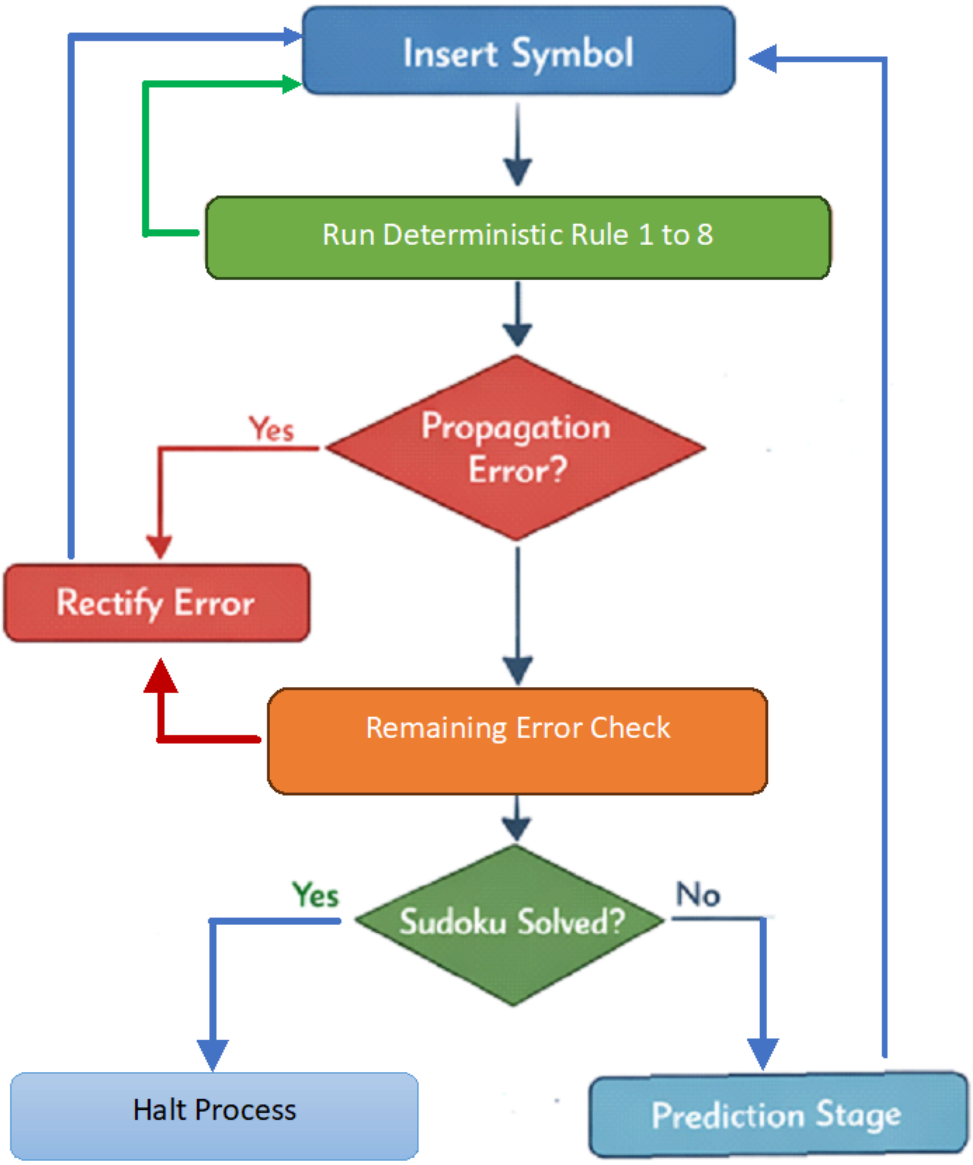
Appendix C — Flow Chart of Algorithm Implementation

The flow chart below illustrates the complete CTST algorithm implementation, showing the pipeline from initialisation through deterministic filtering (Rules 1–8), the prediction phase, error detection, and the rectification and rollback mechanism.

Algorithm Implementation: Solving Sudoku

The algorithm begins by reading the input file and constructing three data structures:

- A puzzle board (two-dimensional table),
- An options table (three-dimensional table), and
- An Error Track Table (three-dimensional table).


Insert Symbol
Run Deterministic Rule 1 to 8
Propagation Error?
Yes
Rectify Error
Remaining Error Check
Sudoku Solved?
Yes
No
Halt Process
Prediction Stage